

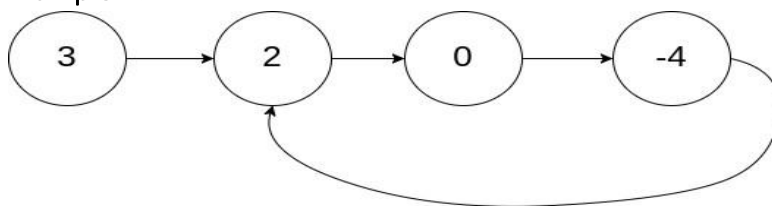
Problem 142 Linked List Cycle-2

Given the head of a linked list, return *the node where the cycle begins*. If there is no cycle, return null.

There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the next pointer. Internally, pos is used to denote the index of the node that tail's next pointer is connected to (0-indexed). It is -1 if there is no cycle. Note that pos is not passed as a parameter.

Do not modify the linked list.

Example 1:

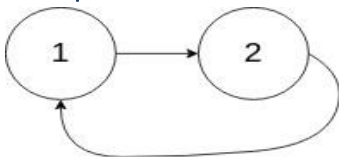


Input: head = [3,2,0,-4], pos = 1

Output: tail connects to node index 1

Explanation: There is a cycle in the linked list, where tail connects to the second node.

Example 2:

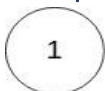


Input: head = [1,2], pos = 0

Output: tail connects to node index 0

Explanation: There is a cycle in the linked list, where tail connects to the first node.

Example 3:



Input: head = [1], pos = -1

Output: no cycle

Explanation: There is no cycle in the linked list.

Constraints:

- The number of the nodes in the list is in the range $[0, 10^4]$.
- $-10^5 \leq \text{Node.val} \leq 10^5$

- pos is -1 or a valid index in the linked-list.

Solution

```
/**
```

```
* Definition for singly-linked list.
```

```
* struct ListNode {
```

```
*   int val;
```

```
*   ListNode *next;
```

```
*   ListNode(int x) : val(x), next(NULL) {}
```

```
* };
```

```
*/
```

```
class Solution {
```

```
public:
```

```
    ListNode *detectCycle(ListNode *head) {
```

```
        if (head == NULL || head->next == NULL)
```

```
            return NULL;
```

```
        ListNode *slow = head;
```

```
        ListNode *fast = head;
```

```
        while (fast != NULL && fast->next != NULL) {
```

```
            slow = slow->next;
```

```
            fast = fast->next->next;
```

```
            if (slow == fast)
```

```
                break; }
```

```
        if (fast == NULL || fast->next == NULL)
```

```
            return NULL;
```

```
        slow = head;
```

```
        while (slow != fast) {
```

```
            slow = slow->next;
```

```
            fast = fast->next; }
```

```
        return slow; };
```

142. Linked List Cycle II Solved

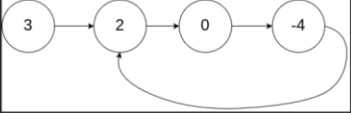
Medium Topics Companies

Given the `head` of a linked list, return the node where the cycle begins. If there is no cycle, return `null`.

There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the `next` pointer. Internally, `pos` is used to denote the index of the node that tail's `next` pointer is connected to (0-indexed). It is `-1` if there is no cycle. Note that `pos` is not passed as a parameter.

Do not modify the linked list.

Example 1:



```

1 /**
2  * Definition for singly-linked list.
3  * struct ListNode {
4  *     int val;
5  *     ListNode *next;
6  *     ListNode(int x) : val(x), next(NULL) {}
7  * };
8  */
9 class Solution {
10 public:
11     ListNode *detectCycle(ListNode *head) {
12         if (head == NULL || head->next == NULL)

```

Accepted 18 / 18 testcases passed
AnshikaPan... submitted at Jan 22, 2026 20:44

Runtime
11 ms | Beats 17.13%

Memory
11.47 MB | Beats 23.69%

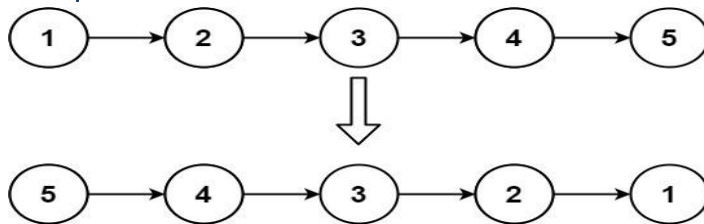
Testcase Test Result
Accepted Runtime: 2 ms
Case 1 Case 2 Case 3

Input
head = [3,2,0,-4]
pos = 1

Problem 206. Reverse Linked List

Given the head of a singly linked list, reverse the list, and return *the reversed list*.

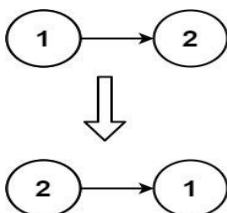
Example 1:



Input: head = [1,2,3,4,5]

Output: [5,4,3,2,1]

Example 2:



Input: head = [1,2]

Output: [2,1]

Example 3:

Input: head = []

Output: []

Constraints:

- The number of nodes in the list is the range [0, 5000].

- $-5000 \leq \text{Node.val} \leq 5000$

Solution:

```
/**  
 * Definition for singly-linked list.  
 * struct ListNode {  
 *     int val;  
 *     ListNode *next;  
 *     ListNode() : val(0), next(nullptr) {}  
 *     ListNode(int x) : val(x), next(nullptr) {}  
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}  
 * };  
 */
```

```
class Solution {
```

```
public:
```

```
    ListNode* reverseList(ListNode* head) {
```

```
        ListNode* prev = NULL;
```

```
        ListNode* curr = head;
```

```
        while (curr != NULL) {
```

```
            ListNode* nextNode = curr->next;
```

```
            curr->next = prev;
```

```
            prev = curr;
```

```
            curr = nextNode;}
```

```
        return prev;
```

```
};
```

206. Reverse Linked List Solved

Given the `head` of a singly linked list, reverse the list, and return the *reversed list*.

Example 1:

Input: `head = [1,2,3,4,5]`
Output: `[5,4,3,2,1]`

Example 2:

Input: `head = [1,2]`
Output: `[2,1]`

```

1  2  3  4  5
  ↓
5  4  3  2  1

```

```

1  2
  ↓
2  1

```

Code:

```

1  /**
2   * Definition for singly-linked list.
3   * struct ListNode {
4   *     int val;
5   *     ListNode *next;
6   *     ListNode() : val(0), next(nullptr) {}
7   *     ListNode(int x) : val(x), next(nullptr) {}
8   *     ListNode(int x, ListNode *next) : val(x), next(next) {}
9   * };
10
11  class Solution {
12  public:

```

Testcase: Accepted Runtime: 0 ms

Test Result: Accepted

Runtime: 0 ms Beats 100.00%

Memory: 13.43 MB Beats 40.77%

Implementation of Doubly Linked List:

```
#include <iostream>
```

```
using namespace std;
```

```
// Node structure
```

```
struct Node {
```

```
    int data;
```

```
    Node* prev;
```

```
    Node* next;
```

```
    Node(int val) {
```

```
        data = val;
```

```
        prev = NULL;
```

```
        next = NULL;
```

```
    }
```

```
};
```

```
class DoublyLinkedList {
```

```
private:
```

```
    Node* head;
```

public:

```
DoublyLinkedList() {  
    head = NULL;  
}  
// Insert at beginning  
void insertAtBeginning(int val) {  
    Node* newNode = new Node(val);  
    if (head != NULL) {  
        newNode->next = head;  
        head->prev = newNode;  
    }  
    head = newNode;  
}  
  
// Insert at end  
void insertAtEnd(int val) {  
    Node* newNode = new Node(val);  
    if (head == NULL) {  
        head = newNode;  
        return;  
    }  
    Node* temp = head;  
    while (temp->next != NULL)  
        temp = temp->next;  
    temp->next = newNode;  
    newNode->prev = temp;  
}  
  
// Insert at position (1-based index)  
void insertAtPosition(int val, int pos) {
```

```
if (pos == 1) {
    insertAtBeginning(val);
    return;
}
Node* temp = head;
for (int i = 1; temp != NULL && i < pos - 1; i++)
    temp = temp->next;
if (temp == NULL) {
    cout << "Invalid position\n";
    return;
}
Node* newNode = new Node(val);
newNode->next = temp->next;
newNode->prev = temp;
if (temp->next != NULL)
    temp->next->prev = newNode;
temp->next = newNode;
}
```

```
// Delete from beginning
void deleteFromBeginning() {
    if (head == NULL) {
        cout << "List is empty\n";
        return;
    }
    Node* temp = head;
    head = head->next;
    if (head != NULL)
        head->prev = NULL;
}
```

```
    delete temp;
}
```

```
// Delete from end
void deleteFromEnd() {
    if (head == NULL) {
        cout << "List is empty\n";
        return;
    }
    Node* temp = head;
    while (temp->next != NULL)
        temp = temp->next;
    if (temp->prev != NULL)
        temp->prev->next = NULL;
    else
        head = NULL;
    delete temp;
}
```

```
// Delete by value
void deleteByValue(int val) {
    Node* temp = head;
    while (temp != NULL && temp->data != val)
        temp = temp->next;
    if (temp == NULL) {
        cout << "Value not found\n";
        return;
    }
    if (temp->prev != NULL)
        temp->prev->next = temp->next;
```



```
else
    head = temp->next;
if (temp->next != NULL)
    temp->next->prev = temp->prev;
delete temp;
}
```

```
// Display
void display() {
    Node* temp = head;
    cout << "Forward: ";
    while (temp != NULL) {
        cout << temp->data << " ";
        temp = temp->next;
    }
    cout << endl;
}
```